ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ
ДО ЛАБОРАТОРНОЇ РОБОТИ №6

«Автоматизоване тестування Python-програм»

з курсу «Професійний Python»
Варіант №3 — «Інтернет-магазин»

Виконав(ла): Вересюк Ірина

Група: ФЕП-32

Перевірив(ла):Куньо І.М.

Дата: 27.10.26

Львів — 2026

1. Мета роботи

Сформувати практичні навички професійного автоматизованого тестування Python-програм із застосуванням фреймворку pytest. Навчитися проєктувати модульні (unit) та інтеграційні (integration) тести, створювати фікстури (fixtures), застосовувати параметризацію, тестувати викид очікуваних винятків (pytest.raises), ізолювати файлову систему за допомогою tmp_path та конфігурацію оточення за допомогою monkeypatch. Опанувати роботу з тестовими двійниками (Mock, AsyncMock), перевіряти стан та взаємодію компонентів, а також виконувати детальний аналіз покриття коду тестами (statement та branch coverage) за допомогою pytest-cov.

2. Індивідуальний варіант і постановка задачі

Варіант №3: «Інтернет-магазин».

Потрібно реалізувати та протестувати такі функціональні вимоги:

1. Створення сутності Product та бізнес-валідація полів price і quantity.
2. Додавання товару до каталогу, пошук товару за ID та фільтрація списку за category.
3. Розрахунок повної вартості складських залишків та визначення найдорожчого товару каталогу.
4. Перевірка граничних (boundary) та некоректних (invalid) значень числових полів.
5. Обробка сценарію взаємодії з порожнім каталогом.
6. Імпорт товарів із CSV-файлу та обробка пошкодженого запису (malformed product record).
7. Серіалізація та експорт структури даних у формат JSON.
8. Зчитування та перевірка конфігурації мінімального порогу залишків (min_stock_threshold) із середовища.
9. Ізоляція сховища через InMemoryProductRepository (Fake) та Mock.
10. Підміна політики знижок/повідомлень через Mock із налаштуванням return_value та перевіркою виклику.
11. Взаємодія з асинхронним зовнішнім постачальником залишків через AsyncMock із перевіркою успішного шляху та аварійного сценарію (side_effect).
12. Побудова наскрізного інтеграційного конвеєра: import → processing → export.
13. Забезпечення сумарного покриття коду тестами понад 85% із контролем покриття гілок (branch coverage).

3. Структура проєкту

Проєкт організовано за сучасною модульною архітектурою src-layout з ізоляцією тестової підсистеми:

online_store_testing/

├── .gitignore

├── pyproject.toml

```
├── README.md
├── report/
│   └── Вересюк_Ірина_Лаб06_V3.pdf
├── src/
│   └── shop/
│       ├── __init__.py
│       ├── config.py
│       ├── exceptions.py
│       ├── exporters.py
│       ├── models.py
│       ├── protocols.py
│       └── services.py
└── tests/
    ├── __init__.py
    ├── conftest.py
    ├── integration/
    │   ├── test_import_export.py
    │   └── test_pipeline.py
    └── unit/
        ├── test_config.py
        ├── test_models.py
        └── test_services.py
```

4. Опис створених модулів та лістинги коду

4.1. src/shop/exceptions.py

Модуль містить ієрархію власних винятків предметної області, що дозволяє точно розділяти типи помилок при обробці замовлень і товарів.

class ShopException(Exception):

```
    """Базовий виняток інтернет-магазину."""


class ValidationError(ShopException):
    """Помилка валідації вхідних даних чи стану об'єкта."""


class MalformedRecordError(ShopException):
    """Помилка некоректного формату запису при завантаженні/імпорті."""


class ProductNotFoundError(ShopException):
    """Виняток, коли запитуваний товар відсутній у каталозі."""


class EmptyCatalogError(ShopException):
    """Виняток при спробі виконання операцій над порожнім каталогом."""


class ExternalProviderError(ShopException):
    """Помилка взаємодії із зовнішнім провайдером залишків."""
```

4.2. src/shop/models.py

Інкапсулює незмінну сутність товару Product, створену за допомогою @dataclass(frozen=True, slots=True). У методі __post_init__ перевіряється обов'язкова непорожність назви, а також невід'ємність числових показників вартості та кількості.

```
from dataclasses import dataclass

from shop.exceptions import ValidationError


@dataclass(frozen=True, slots=True)

class Product:

    product_id: int

    name: str
```

```
    category: str
    price: float
    quantity: int

    def __post_init__(self) -> None:
        if self.price < 0:
            raise ValidationError(f"Ціна не може бути від'ємною: {self.price}")
        if self.quantity < 0:
            raise ValidationError(f"Кількість не може бути від'ємною: {self.quantity}")
        if not self.name.strip():
            raise ValidationError("Назва товару не може бути порожньою")

    @property
    def total_cost(self) -> float:
        """Вартість залишків конкретного товару на складі."""
        return self.price * self.quantity
```

4.3. src/shop/protocols.py

Визначає контракти взаємодії (інтерфейси) за допомогою typing.Protocol, що дозволяє ізолювати залежності під час тестування через Mocks та Fakes.

```
from typing import Protocol
from shop.models import Product

class ProductRepository(Protocol):
    """Інтерфейс для сховища товарів."""
    def get_by_id(self, product_id: int) -> Product | None:
        ...
```

```
    def get_all(self) -> list[Product]:
        ...

    def add(self, product: Product) -> None:
        ...

class DiscountPolicy(Protocol):
    """Інтерфейс для розрахунку знижок."""
    def apply_discount(self, total: float) -> float:
        ...

class ExternalInventoryProvider(Protocol):
    """Асинхронний інтерфейс зовнішнього постачальника."""
    async def fetch_stock_update(self, product_id: int) -> int:
        ...
```

4.4. src/shop/services.py

Реалізує бізнес-логіку каталогу: додавання, пошук, фільтрацію за категорією, розрахунок загальної вартості запасів із можливістю застосування політики знижок, обчислення максимального елемента та асинхронне оновлення залишків через зовнішнього постачальника.

```
from shop.exceptions import EmptyCatalogError, ProductNotFoundError,
ValidationError
from shop.models import Product
from shop.protocols import DiscountPolicy, ExternalInventoryProvider,
ProductRepository

class CatalogService:
    def __init__(self, repository: ProductRepository) -> None:
        self._repo = repository
```

```
    def add_product(self, product: Product) -> None:
        self._repo.add(product)

    def find_product(self, product_id: int) -> Product:
        item = self._repo.get_by_id(product_id)
        if item is None:
            raise ProductNotFoundError(f"Товар з ID {product_id} не знайдено.")
        return item

    def filter_by_category(self, category: str) -> list[Product]:
        return [p for p in self._repo.get_all() if p.category.lower() == category.lower()]

    def calculate_total_inventory_value(self, discount_policy: DiscountPolicy | None =
None) -> float:
        items = self._repo.get_all()
        total = sum(p.total_cost for p in items)
        if discount_policy is not None:
            return discount_policy.apply_discount(total)
        return total

    def get_most_expensive_product(self) -> Product:
        items = self._repo.get_all()
        if not items:
            raise EmptyCatalogError("Каталог порожній")
        return max(items, key=lambda p: p.price)
```

```
    async def sync_stock_with_provider(
        self, product_id: int, provider: ExternalInventoryProvider
    ) -> Product:
        current_product = self.find_product(product_id)
        new_quantity = await provider.fetch_stock_update(product_id)
        if new_quantity < 0:
            raise ValidationError("Провайдер повернув некоректну кількість")


        updated = Product(
            product_id=current_product.product_id,
            name=current_product.name,
            category=current_product.category,
            price=current_product.price,
            quantity=new_quantity,
        )
        self._repo.add(updated)
        return updated
```

4.5. src/shop/exporters.py

Забезпечує операції файлового введення/виведення: читання та валідацію рядків CSV-таблиці з викидом MalformedRecordError при некоректних даних, а також експорт списку об'єктів у структурований JSON.

```
import csv
import json
from pathlib import Path
from shop.exceptions import MalformedRecordError
from shop.models import Product
```

```
def import_products_from_csv(file_path: Path) -> list[Product]:
    products: list[Product] = []
    if not file_path.exists():
        raise FileNotFoundError(f"Файл не знайдено: {file_path}")

    with file_path.open("r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row_idx, row in enumerate(reader, start=2):
            try:
                products.append(
                    Product(
                        product_id=int(row["id"]),
                        name=row["name"],
                        category=row["category"],
                        price=float(row["price"]),
                        quantity=int(row["quantity"]),
                    )
                )
            except (KeyError, ValueError, TypeError) as err:
                raise MalformedRecordError(
                    f"Помилка у рядку {row_idx}: некоректні дані"
                ) from err
    return products

def export_products_to_json(products: list[Product], output_path: Path) -> None:
    data = [
```

```
        {
            "id": p.product_id,
            "name": p.name,
            "category": p.category,
            "price": p.price,
            "quantity": p.quantity,
            "total_cost": p.total_cost,
        }
        for p in products
    ]
    output_path.write_text(json.dumps(data, indent=2, ensure_ascii=False),
encoding="utf-8")
```

4.6. src/shop/config.py

Зчитує конфігураційний параметр мінімального порогу залишків на складі із середовища (MIN_STOCK_THRESHOLD) із безпечним поверненням значення за замовчуванням.

```
import os


def get_min_stock_threshold() -> int:
    """Повертає мінімальний поріг залишків на складі із середовища (env var)."""
    val = os.getenv("MIN_STOCK_THRESHOLD", "5")
    try:
        threshold = int(val)
        if threshold < 0:
            return 5
        return threshold
    except ValueError:
        return 5
```

5. Опис тестового набору та фікстур (tests/)

5.1. tests/conftest.py

Містить спільні тестові фікстури та реалізацію Fake-об'єкта InMemoryProductRepository, що дозволяє ізолювати тести від зовнішніх баз даних без оверинжинірингу.

```
import pytest

from shop.models import Product

from shop.protocols import ProductRepository


class InMemoryProductRepository(ProductRepository):

    """Fake-репозиторій для ізольованого тестування логіки без зовнішніх БД."""

    def __init__(self) -> None:

        self._storage: dict[int, Product] = {}


    def get_by_id(self, product_id: int) -> Product | None:

        return self._storage.get(product_id)


    def get_all(self) -> list[Product]:

        return list(self._storage.values())


    def add(self, product: Product) -> None:

        self._storage[product.product_id] = product


@pytest.fixture

def empty_repo() -> InMemoryProductRepository:

    return InMemoryProductRepository()
```

```
@pytest.fixture
def sample_products() -> list[Product]:
    return [
        Product(1, "Ноутбук", "Електроніка", 25000.0, 3),
        Product(2, "Мишка", "Електроніка", 500.0, 10),
        Product(3, "Кава", "Продукти", 250.0, 20),
    ]


@pytest.fixture
def populated_repo(
    empty_repo: InMemoryProductRepository, sample_products: list[Product]
) -> InMemoryProductRepository:
    for item in sample_products:
        empty_repo.add(item)
    return empty_repo
```

5.2. tests/unit/test_models.py

Перевіряє коректне створення об'єкта, розрахунок загальної вартості залишків за допомогою pytest.approx, поведінку на межах допустимих значень (0.0, 0.01) через @pytest.mark.parametrize та валідаційні винятки pytest.raises.

```
import pytest
from shop.exceptions import ValidationError
from shop.models import Product


def test_product_creation_valid():
    p = Product(1, "Стіл", "Меблі", 1500.0, 4)
    assert p.product_id == 1
    assert p.name == "Стіл"
```

```
    assert p.category == "Меблі"
    assert p.total_cost == pytest.approx(6000.0)


@pytest.mark.parametrize(
    "price, quantity",
    [
        (0.0, 0),
        (0.01, 1),
        (100.0, 9999),
    ],
    ids=["zero_boundary", "min_positive_boundary", "standard_values"],
)
def test_product_boundary_values(price, quantity):
    p = Product(1, "Товар", "Категорія", price, quantity)
    assert p.price == pytest.approx(price)
    assert p.quantity == quantity


@pytest.mark.parametrize(
    "price, quantity, name, error_match",
    [
        (-0.01, 5, "Товар", "Ціна не може бути від'ємною"),
        (-100.0, 5, "Товар", "Ціна не може бути від'ємною"),
        (100.0, -1, "Товар", "Кількість не може бути від'ємною"),
        (100.0, 5, "   ", "Назва товару не може бути порожньою"),
    ],
    ids=["negative_boundary_price", "negative_price", "negative_quantity",
"blank_name"],
```

```
)

def test_product_validation_errors(price, quantity, name, error_match):

    with pytest.raises(ValidationError, match=error_match):

        Product(1, name, "Категорія", price, quantity)
```

5.3. tests/unit/test_services.py

Тестує бізнес-операції сервісу, взаємодію з інтерфейсом репозиторію через Mock, застосування знижки, поведінку порожньої колекції та асинхронні виклики через AsyncMock (зокрема обробку успішного оновлення і збою через side_effect).

```
from unittest.mock import AsyncMock, Mock

import pytest

from shop.exceptions import (

    EmptyCatalogError,

    ExternalProviderError,

    ProductNotFoundError,

    ValidationError,

)

from shop.models import Product

from shop.protocols import DiscountPolicy, ExternalInventoryProvider,
ProductRepository

from shop.services import CatalogService


def test_add_and_find_product(empty_repo):

    service = CatalogService(empty_repo)

    product = Product(10, "Клавіатура", "Електроніка", 1200.0, 5)

    service.add_product(product)


    found = service.find_product(10)

    assert found == product
```

```
def test_find_product_not_found_raises(empty_repo):
    service = CatalogService(empty_repo)
    with pytest.raises(ProductNotFoundError, match="не знайдено"):
        service.find_product(999)


def test_filter_by_category(populated_repo):
    service = CatalogService(populated_repo)
    electronics = service.filter_by_category("Електроніка")
    assert len(electronics) == 2
    assert all(p.category == "Електроніка" for p in electronics)


def test_filter_by_category_case_insensitive(populated_repo):
    service = CatalogService(populated_repo)
    food = service.filter_by_category("продукти")
    assert len(food) == 1
    assert food[0].name == "Кава"


def test_calculate_total_inventory_value_without_discount(populated_repo):
    service = CatalogService(populated_repo)
    # 25000*3 + 500*10 + 250*20 = 75000 + 5000 + 5000 = 85000
    assert service.calculate_total_inventory_value() == pytest.approx(85000.0)


def test_calculate_total_inventory_with_discount_mock(populated_repo):
    service = CatalogService(populated_repo)
    discount_mock = Mock(spec=DiscountPolicy)
```

```
    discount_mock.apply_discount.return_value = 76500.0

    result = service.calculate_total_inventory_value(discount_policy=discount_mock)

    assert result == pytest.approx(76500.0)
    discount_mock.apply_discount.assert_called_once_with(85000.0)

def test_most_expensive_product(populated_repo):
    service = CatalogService(populated_repo)
    expensive = service.get_most_expensive_product()
    assert expensive.name == "Ноутбук"
    assert expensive.price == pytest.approx(25000.0)

def test_most_expensive_product_empty_catalog_raises(empty_repo):
    service = CatalogService(empty_repo)
    with pytest.raises(EmptyCatalogError, match="Каталог порожній"):
        service.get_most_expensive_product()

def test_empty_catalog_total_value(empty_repo):
    service = CatalogService(empty_repo)
    assert service.calculate_total_inventory_value() == 0.0

def test_repository_mock_interaction():
    mock_repo = Mock(spec=ProductRepository)
    mock_repo.get_by_id.return_value = Product(1, "Тест", "Кат", 100.0, 1)
```

```
    service = CatalogService(mock_repo)
    p = service.find_product(1)

    assert p.name == "Тест"
    mock_repo.get_by_id.assert_called_once_with(1)

@pytest.mark.asyncio
async def test_sync_stock_with_provider_success(empty_repo):
    product = Product(1, "Монітор", "Електроніка", 8000.0, 2)
    empty_repo.add(product)
    service = CatalogService(empty_repo)

    mock_provider = AsyncMock(spec=ExternalInventoryProvider)
    mock_provider.fetch_stock_update.return_value = 15

    updated = await service.sync_stock_with_provider(1, mock_provider)

    assert updated.quantity == 15
    mock_provider.fetch_stock_update.assert_awaited_once_with(1)

@pytest.mark.asyncio
async def test_sync_stock_with_provider_side_effect_failure(empty_repo):
    product = Product(1, "Монітор", "Електроніка", 8000.0, 2)
    empty_repo.add(product)
    service = CatalogService(empty_repo)
```

```
    mock_provider = AsyncMock(spec=ExternalInventoryProvider)
    mock_provider.fetch_stock_update.side_effect = ExternalProviderError("Network Down")

    with pytest.raises(ExternalProviderError, match="Network Down"):
        await service.sync_stock_with_provider(1, mock_provider)

@pytest.mark.asyncio
async def test_sync_stock_with_invalid_provider_response(empty_repo):
    product = Product(1, "Монітор", "Електроніка", 8000.0, 2)
    empty_repo.add(product)
    service = CatalogService(empty_repo)

    mock_provider = AsyncMock(spec=ExternalInventoryProvider)
    mock_provider.fetch_stock_update.return_value = -3

    with pytest.raises(ValidationError, match="некоректну кількість"):
        await service.sync_stock_with_provider(1, mock_provider)
```

5.4. tests/unit/test_config.py

Демонструє ізоляцію середовища за допомогою monkeypatch.

```
from shop.config import get_min_stock_threshold

def test_min_stock_threshold_custom_env(monkeypatch):
    monkeypatch.setenv("MIN_STOCK_THRESHOLD", "12")
    assert get_min_stock_threshold() == 12

def test_min_stock_threshold_default(monkeypatch):
```

```
    monkeypatch.delenv("MIN_STOCK_THRESHOLD", raising=False)
    assert get_min_stock_threshold() == 5


def test_min_stock_threshold_invalid_string(monkeypatch):
    monkeypatch.setenv("MIN_STOCK_THRESHOLD", "not_a_number")
    assert get_min_stock_threshold() == 5


def test_min_stock_threshold_negative_env(monkeypatch):
    monkeypatch.setenv("MIN_STOCK_THRESHOLD", "-7")
    assert get_min_stock_threshold() == 5
```

5.5. tests/integration/test_import_export.py

Перевіряє операції з файлами в ізольованій тимчасовій директорії за допомогою фікстури tmp_path.

```
import json
import pytest
from shop.exceptions import MalformedRecordError
from shop.exporters import export_products_to_json, import_products_from_csv
from shop.models import Product


@pytest.mark.integration
def test_csv_import_success(tmp_path):
    csv_file = tmp_path / "products.csv"
    csv_file.write_text(
        "id,name,category,price,quantity\n"
        "1,Чайник,Побутова техніка,1200.5,5\n"
        "2,Праска,Побутова техніка,2500.0,2\n",
        encoding="utf-8",
```

```
    )

    items = import_products_from_csv(csv_file)

    assert len(items) == 2
    assert items[0].name == "Чайник"
    assert items[0].price == pytest.approx(1200.5)

@pytest.mark.integration
def test_csv_import_malformed_record_raises(tmp_path):
    csv_file = tmp_path / "malformed.csv"
    csv_file.write_text(
        "id,name,category,price,quantity\n"
        "1,Чайник,Побутова техніка,bad_price,5\n",
        encoding="utf-8",
    )

    with pytest.raises(MalformedRecordError, match="Помилка у рядку 2"):
        import_products_from_csv(csv_file)

@pytest.mark.integration
def test_csv_import_missing_file_raises(tmp_path):
    missing_file = tmp_path / "non_existing.csv"
    with pytest.raises(FileNotFoundError):
        import_products_from_csv(missing_file)
```

```
@pytest.mark.integration
def test_export_and_read_json(tmp_path):
    json_file = tmp_path / "export.json"
    products = [
        Product(10, "Навушники", "Аудіо", 1500.0, 2),
    ]

    export_products_to_json(products, json_file)

    assert json_file.exists()
    data = json.loads(json_file.read_text(encoding="utf-8"))
    assert len(data) == 1
    assert data[0]["name"] == "Навушники"
    assert data[0]["total_cost"] == pytest.approx(3000.0)
```

5.6. tests/integration/test_pipeline.py

Наскрізний інтеграційний тест життєвого циклу даних: читання з CSV → реєстрація в каталозі → асинхронне оновлення через провайдера → серіалізація в JSON-файл.

```
import json
from unittest.mock import AsyncMock
import pytest
from shop.exporters import export_products_to_json, import_products_from_csv
from shop.protocols import ExternalInventoryProvider
from shop.services import CatalogService

@pytest.mark.integration
@pytest.mark.asyncio
```

```
async def test_full_shop_lifecycle_pipeline(tmp_path, empty_repo):
    # 1. Створення тимчасового CSV із тестовими даними
    input_csv = tmp_path / "catalog_in.csv"
    output_json = tmp_path / "catalog_out.json"
    input_csv.write_text(
        "id,name,category,price,quantity\n"
        "1,Планшет,Гаджети,12000.0,2\n"
        "2,Чохол,Аксесуари,500.0,10\n",
        encoding="utf-8",
    )

    # 2. Імпорт товарів і збереження у сховищі (використовуємо empty_repo як InMemoryProductRepository)
    items = import_products_from_csv(input_csv)
    service = CatalogService(empty_repo)
    for product in items:
        service.add_product(product)

    # 3. Асинхронна синхронізація залишків через Mock
    mock_provider = AsyncMock(spec=ExternalInventoryProvider)
    mock_provider.fetch_stock_update.return_value = 5  # оновлено залишок для ID 1
    await service.sync_stock_with_provider(1, mock_provider)

    # 4. Експорт оновленого каталогу в JSON
    all_items = empty_repo.get_all()
    export_products_to_json(all_items, output_json)
```

```
    # 5. Верифікація результатів згенерованого файлу
    exported_data = json.loads(output_json.read_text(encoding="utf-8"))
    tablet_item = next(p for p in exported_data if p["id"] == 1)

    assert tablet_item["quantity"] == 5
    assert tablet_item["total_cost"] == pytest.approx(60000.0)
    assert service.get_most_expensive_product().name == "Планшет"
```

6. Файл pyproject.toml

Конфігураційний файл проєкту задає систему збірки setuptools, параметризацію pytest, опцію автоматичної підтримки асинхронності asyncio_mode = "auto", реєстрацію маркерів та поріг покриття коду fail_under = 85 із розрахунком покриття розгалужень (branch = true).

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "online-store-testing"
version = "0.1.0"
description = "Лабораторна робота №6. Автоматизоване тестування Python-програм (Варіант 3)"
readme = "README.md"
requires-python = ">=3.10"
dependencies = []

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
```

```
    "pytest-cov>=5.0.0",
    "pytest-asyncio>=0.23.0",
]

[tool.pytest.ini_options]
minversion = "8.0"
testpaths = ["tests"]
pythonpath = [".", "src"]
asyncio_mode = "auto"
markers = [
    "integration: integration tests pipeline",
]

[tool.coverage.run]
source = ["shop"]
branch = true

[tool.coverage.report]
show_missing = true
skip_covered = false
fail_under = 85
```

7. Результати запуску тестів та звіт покриття

7.1. Фактичне виведення у терміналі PowerShell

```
(.venv) PS D:\Робочий стіл\проф. python\online_store_testing> pytest -v --cov=shop --cov-branch --cov-report=term-missing --cov-report=html

====================== test session starts ======================
```

platform win32 -- Python 3.13.0, pytest-9.1.1, pluggy-1.7.0 -- D:\Робочий стіл\проф. python\online_store_testing\.venv\Scripts\python.exe

cachedir: .pytest_cache

rootdir: D:\Робочий стіл\проф. python\online_store_testing

configfile: pyproject.toml

testpaths: tests

plugins: asyncio-1.4.1, cov-7.1.0

asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function

collected 30 items

tests/integration/test_import_export.py::test_csv_import_success PASSED [3%]

tests/integration/test_import_export.py::test_csv_import_malformed_record_raises PASSED [6%]

tests/integration/test_import_export.py::test_csv_import_missing_file_raises PASSED [10%]

tests/integration/test_import_export.py::test_export_and_read_json PASSED [13%]

tests/integration/test_pipeline.py::test_full_shop_lifecycle_pipeline PASSED [16%]

tests/unit/test_config.py::test_min_stock_threshold_custom_env PASSED [20%]

tests/unit/test_config.py::test_min_stock_threshold_default PASSED [23%]

tests/unit/test_config.py::test_min_stock_threshold_invalid_string PASSED [26%]

tests/unit/test_config.py::test_min_stock_threshold_negative_env PASSED [30%]

tests/unit/test_models.py::test_product_creation_valid PASSED [33%]

tests/unit/test_models.py::test_product_boundary_values[zero_boundary] PASSED [36%]

tests/unit/test_models.py::test_product_boundary_values[min_positive_boundary] PASSED [40%]

tests/unit/test_models.py::test_product_boundary_values[standard_values] PASSED [43%]

tests/unit/test_models.py::test_product_validation_errors[negative_boundary_price] PASSED [46%]

tests/unit/test_models.py::test_product_validation_errors[negative_price] PASSED [50%]

tests/unit/test_models.py::test_product_validation_errors[negative_quantity] PASSED [53%]

tests/unit/test_models.py::test_product_validation_errors[blank_name] PASSED [56%]

tests/unit/test_services.py::test_add_and_find_product PASSED [60%]

tests/unit/test_services.py::test_find_product_not_found_raises PASSED [63%]

tests/unit/test_services.py::test_filter_by_category PASSED [66%]

tests/unit/test_services.py::test_filter_by_category_case_insensitive PASSED [70%]

tests/unit/test_services.py::test_calculate_total_inventory_value_without_discount PASSED [73%]

tests/unit/test_services.py::test_calculate_total_inventory_with_discount_mock PASSED [76%]

tests/unit/test_services.py::test_most_expensive_product PASSED [80%]

tests/unit/test_services.py::test_most_expensive_product_empty_catalog_raises PASSED [83%]

tests/unit/test_services.py::test_empty_catalog_total_value PASSED [86%]

tests/unit/test_services.py::test_repository_mock_interaction PASSED [90%]

tests/unit/test_services.py::test_sync_stock_with_provider_success PASSED [93%]

tests/unit/test_services.py::test_sync_stock_with_provider_side_effect_failure PASSED [96%]

tests/unit/test_services.py::test_sync_stock_with_invalid_provider_response PASSED [100%]

========== tests coverage ==========

_____ coverage: platform win32, python 3.13.0-final-0 _____

Name	Stmts	Miss	Branch	BrPart	Cover	Missing

---

src\shop__init__.py 0 0 0 0 100%

src\shop\config.py 10 0 2 0 100%

src\shop\exceptions.py 6 0 0 0 100%

src\shop\exporters.py 20 0 4 0 100%

src\shop\models.py 19 0 6 0 100%

src\shop\protocols.py 5 0 0 0 100%

src\shop\services.py 34 0 8 0 100%

TOTAL 94 0 20 0 100%

Coverage HTML written to dir htmlcov

Required test coverage of 85.0% reached. Total coverage: 100.00%

========== 30 passed in 1.18s ==========

7.2. Аналіз покриття коду

За результатами запуску набору тестів усі 30 тестових функцій завершилися успішно (30 passed). Загальне покриття коду за рядками (statement coverage) склало **100%**, а покриття розгалужень (branch coverage) досягло **20 із 20 можливих гілок (100%)** без жодного частково пройденого розгалуження (BrPart = 0). Це свідчить про повну відсутність мертвих або непротестованих гілок коду у розробленому модулі.

8. Робота з Git

У репозиторії проєкту зафіксовано послідовну історію розробки, розділену на три змістовні логічні коміти:

8.1. Коміт 1: Ініціалізація структури та винятки

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_testing (main)

$ git add .gitignore pyproject.toml README.md src/shop/__init__.py src/shop/exceptions.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_testing (main)

$ git commit -m "Initialize project structure, pyproject.toml and domain exceptions"

[main (root-commit) 2f0523b] Initialize project structure, pyproject.toml and domain exceptions

5 files changed, 115 insertions(+)

create mode 100644 .gitignore

create mode 100644 README.md

create mode 100644 pyproject.toml

create mode 100644 src/shop/__init__.py

create mode 100644 src/shop/exceptions.py

8.2. Коміт 2: Реалізація моделей, сервісів та утиліт

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_testing (main)

$ git add src/shop/models.py src/shop/protocols.py src/shop/config.py src/shop/services.py src/shop/exporters.py

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_testing (main)

$ git commit -m "Implement core models, services, protocols and export-import utilities"

[main 431eb8c] Implement core models, services, protocols and export-import utilities

5 files changed, 159 insertions(+)

create mode 100644 src/shop/config.py

create mode 100644 src/shop/exporters.py

create mode 100644 src/shop/models.py

create mode 100644 src/shop/protocols.py

create mode 100644 src/shop/services.py

8.3. Коміт 3: Повний тестовий набір pytest

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_testing (main)

$ git add tests/

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_testing (main)

$ git commit -m "Add full pytest suite with fixtures, unit and integration tests"

[main 08e5cd5] Add full pytest suite with fixtures, unit and integration tests

7 files changed, 331 insertions(+)

create mode 100644 tests/__init__.py

create mode 100644 tests/conftest.py

create mode 100644 tests/integration/test_import_export.py

create mode 100644 tests/integration/test_pipeline.py

create mode 100644 tests/unit/test_config.py

create mode 100644 tests/unit/test_models.py

create mode 100644 tests/unit/test_services.py

8. 4. Результат виконання git status

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_testing (main)

$ git status

On branch main

nothing to commit, working tree clean

8.5. Результат виконання git log --oneline

Admin@DESKTOP-FDJFLEG MINGW64 /d/Робочий стіл/проф. python/online_store_testing (main)

$ git log --oneline

57e0466 (HEAD -> main) Add full pytest suite with fixtures, unit and integration tests

431eb8c Implement core models, services, protocols and export-import utilities

2f0523b Initialize project structure, pyproject.toml and domain exceptions

9. Висновок

У ході лабораторної роботи №6 реалізовано професійний автоматизований набір тестів для модульної системи інтернет-магазину (Варіант №3).

Тестовий набір побудовано за принципом тестової піраміди та розділено на ізольовані рівні unit- і integration-тестування. Було створено 30 автоматизованих тестів, що покривають нормальні, межові (boundary) та аварійні сценарії валідації сутності Product і каталогу.

Використання фікстур conftest.py дозволило винести підготовку тестових наборів та реалізувати Fake-сховище. За допомогою pytest.mark.parametrize протестовано валідаційні обмеження цін та залишків. Ізоляцію файлової системи реалізовано через фікстуру tmp_path, а безпечне тестування конфігураційних параметрів середовища забезпечено фікстурою monkeypatch. Залежності від зовнішніх сервісів було ізольовано за допомогою тестових двійників Mock та AsyncMock з перевіркою викликів і сценарію падіння через side_effect.

Аналіз покриття за допомогою pytest-cov підтвердив 100% statement coverage та 100% branch coverage (20/20 гілок покрито). Усі етапи роботи зафіксовано в системі контролю версій Git трьома послідовними комітами.